



Da Lógica à Tela: Construindo seu Primeiro App Integrado

IntegraSI — Como a lógica de programação funciona no mercado real.

Ministrantes: Carlos Henrique & Gustavo

- 📅 Duração: oficina prática de 3 horas. Objetivo: sair do console e entregar uma interface funcional usando a API do GitHub.

Quem somos



Carlos Santiago (Caos)

Muitos me conhecem como Caos, e pra mim é como a união de coisas distintas em um unico ser.

Full-Stack Engineer Júnior atuo com Java,

Angular, Laravel,

Fundador da comunidade Bahia Dev Hub.

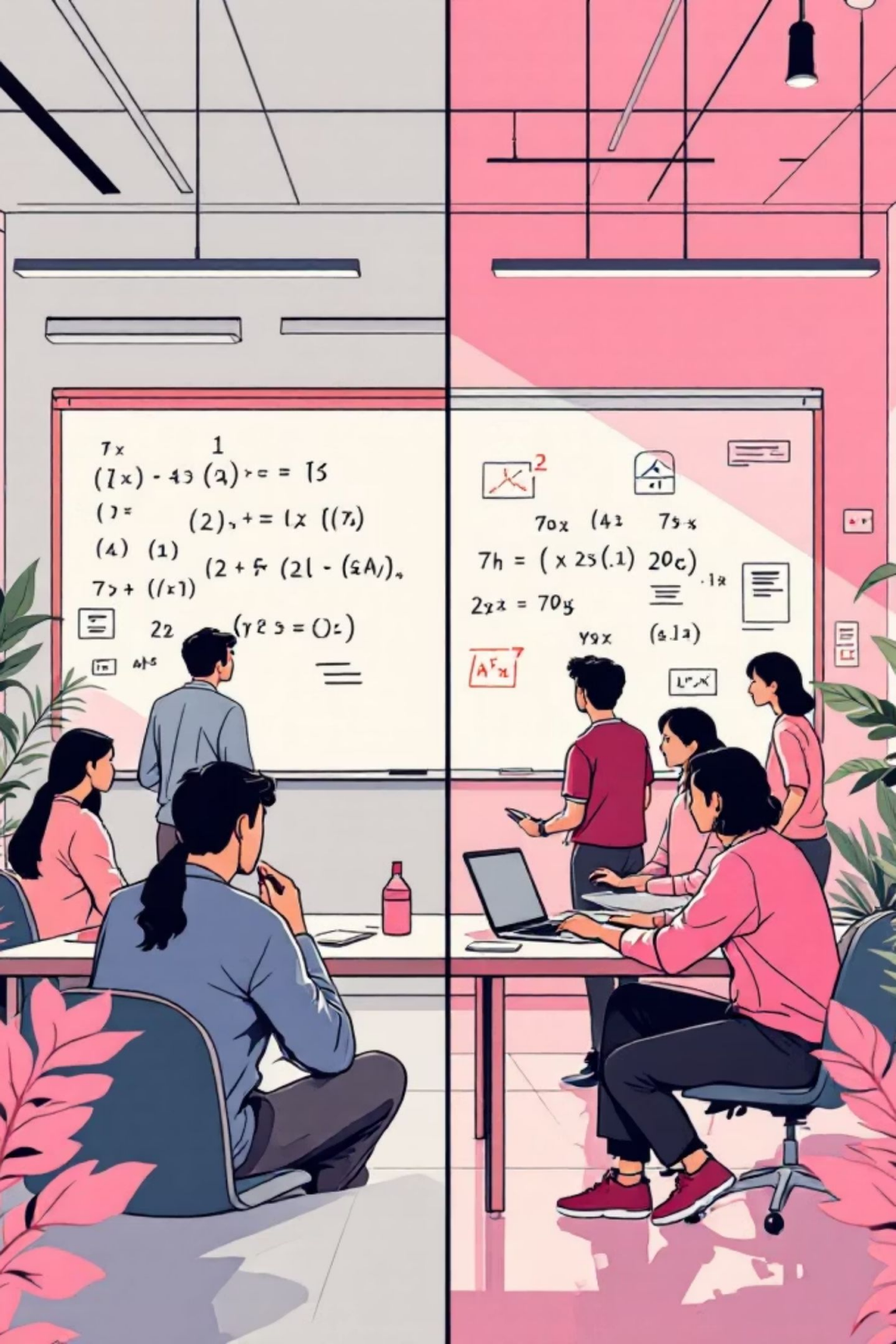
Músico

Entusiasta de Automobilismo



Gustavo Bezerra Nonato

Gustavo é estudante do 5º semestre de Sistemas de Informação, com experiência em diversas linguagens de programação. Tem interesse em desenvolvimento de software, tecnologia e inovação, buscando sempre aprimorar suas habilidades técnicas e construir soluções eficientes e bem estruturadas.



O choque de realidade

Lógica na Faculdade

- Achar número primo
- Imprimir triângulo de asteriscos
- Resolver problemas matemáticos fechados

Lógica no Mercado

- Tratar falhas de rede (Try/Catch)
- Esperar dados assincronamente (Async/Await)
- Desmontar e mapear JSONs

Resumo: hoje a lógica controla fluxo de informação — não só teoria, mas expectativas reais (latência, erros, formato dos dados).



O que é uma API? (Metáfora do restaurante)

Você: o Front-end (cliente) — faz o pedido.

A cozinha: o Banco de Dados/Servidor — prepara os dados.

O garçom: a API — entrega o pedido entre cliente e cozinha.

Antes de codar: entenda o fluxo. Você pede, a API busca, a cozinha responde, e a UI mostra o prato.

A língua da web: JSON

Exemplo (GitHub):

```
{  
  "login": "torvalds",  
  "name": "Linus Torvalds",  
  "followers": 205000,  
  "avatar_url": "https://..."  
}
```

O garçom entrega texto estruturado. Nosso trabalho: ler esse pacote e transformar em interface legível.

Missão de hoje

App

Buscador de Devs do GitHub
— busca por username e
mostra perfil.

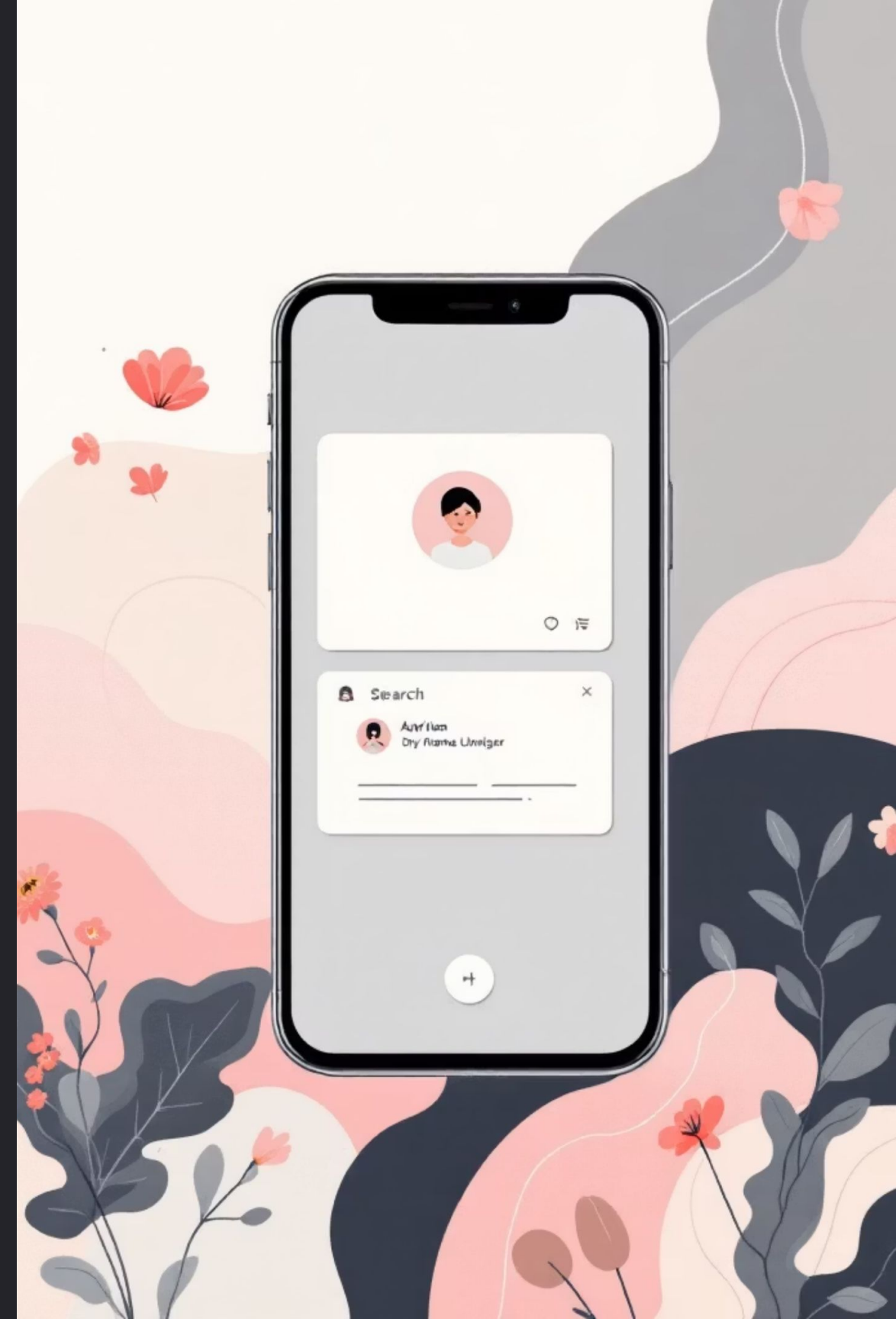
Stack

HTML + JavaScript (fetch) •
Só VS Code e navegador.

Entrega

Input de busca, cartão com foto, nome, repositórios e seguidores.

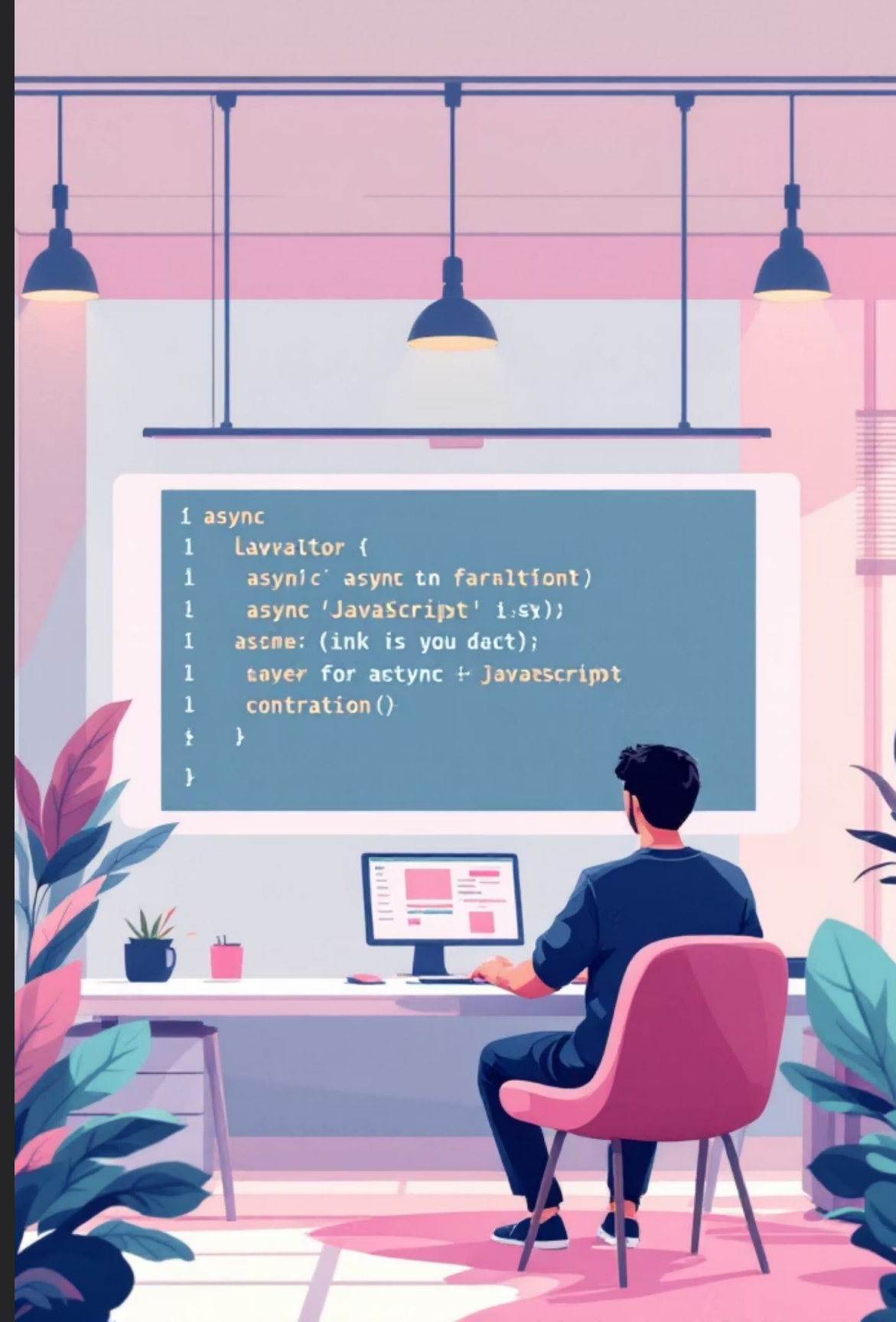
Foco: transformar lógica (fetch + JSON) em elementos do DOM —
rápido e direto.



Passo 1 — O motor lógico (fetch)

```
async function buscarUsuario(nome) {  
  try {  
    const resposta = await  
fetch(`https://api.github.com/users/${nome}`);  
    const dados = await resposta.json();  
    console.log(dados);  
  } catch (erro) {  
    console.error("Deu ruim!");  
  }  
}
```

Dica prática: rode no Console primeiro. Async/await faz o código esperar o garçom voltar — essencial para lidar com latência.

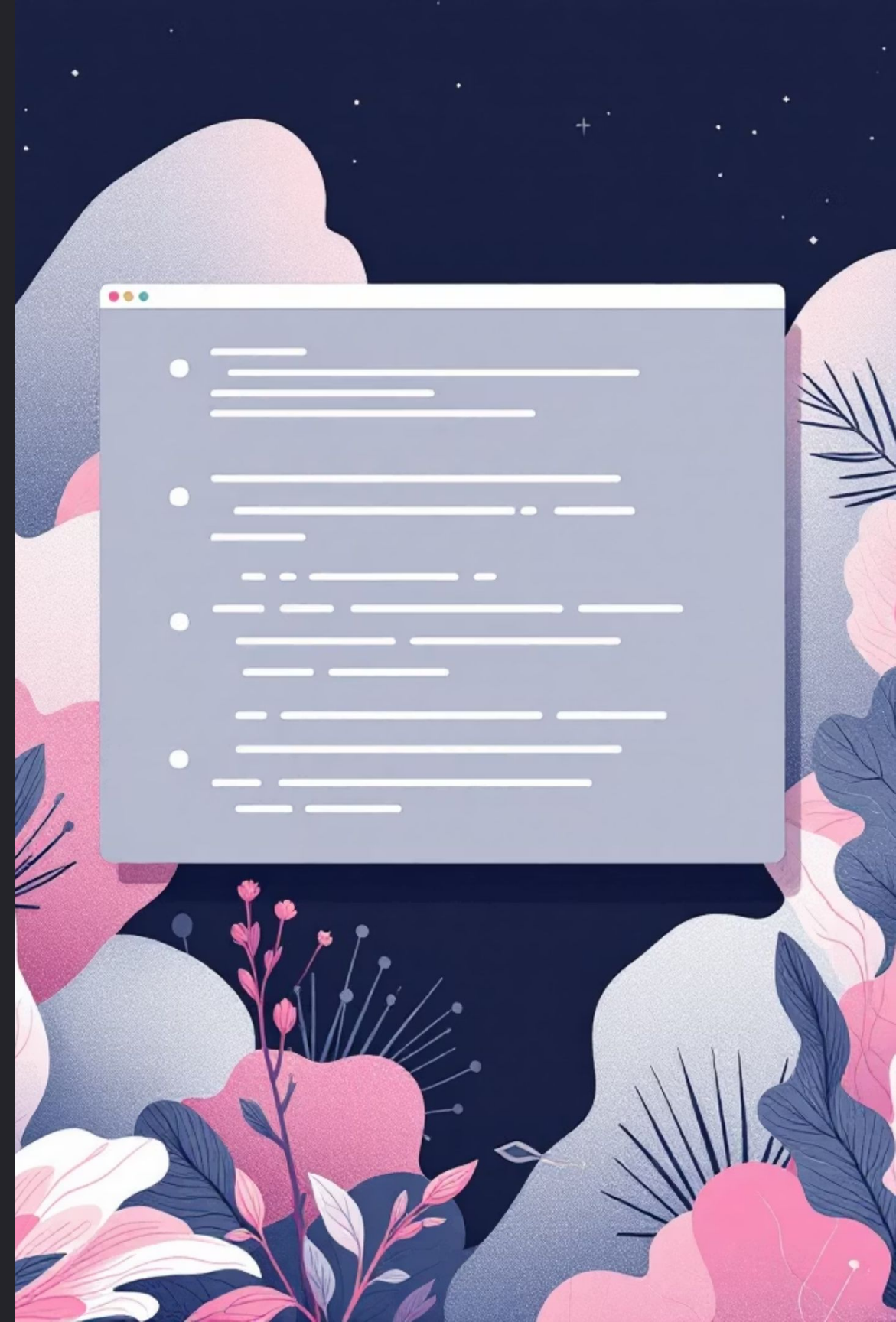


Passo 2 — A interface (HTML)

```
<h1>Buscador do GitHub</h1>
<input type="text" id="inputNome">
<button onclick="acionarBusca()">Buscar</button>

<div id="resultado">
  <img id="fotoPerfil" src="">
  <h2 id="nomeTela"></h2>
</div>
```

Obs: ids são os CPFs das tags — o JS usa esses ids pra localizar e atualizar elementos no DOM.



Passo 3 — Conectando os fios (DOM)

```
// Dentro do Try:
const input = document.getElementById("inputNome").value;
const resposta = await
fetch(`https://api.github.com/users/${input}`);
const dados = await resposta.json();

document.getElementById("fotoPerfil").src =
dados.avatar_url;
document.getElementById("nomeTela").innerText = dados.name;
```

Princípio: mapear propriedades do JSON para atributos/innerText do HTML. Sempre valide antes de usar (dados.name pode ser null).

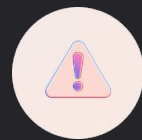


Desafio final + Encerramento



Desafio

Mostrar `public_repos` e `followers` no card lendo o JSON.



Bônus Sênior

Tratar usuário inexistente: exibir `alert()` amigável ou mensagem na tela.



Próximos passos

Melhorar UI, adicionar loading state, debounce na busca e tratamento de rate limit.

Obrigado, IntegraSI! Sigam o Bahia Dev Hub — adicionem a gente no LinkedIn/GitHub. Programar é resolver problemas — vocês já fizeram o primeiro passo.

